

CHƯƠNG 6

QUẢN TRỊ CƠ SỞ DỮ LIỆU

Mục tiêu chương

Sau khi học xong chương này, người học có thể:

- Quản lý người dùng, phân quyền và bảo mật cơ sở dữ liệu SQL.
- Thực hiện backup & restore dữ liệu một cách an toàn.
- Sử dụng công cụ web-based để quản trị DB và file tập trung.
- Tích hợp database vào stack ứng dụng hiện đại.

6.1 Quản lý quyền truy cập và bảo mật CSDL

6.1.1 Cơ bản về quản lý tài khoản người dùng

Quản lý tài khoản người dùng (User Account Management - UAM) trong hệ quản trị cơ sở dữ liệu (DBMS) không chỉ đơn thuần là việc cấp tên đăng nhập và mật khẩu. Đây là một quy trình kỹ thuật và quản lý nhằm xác định, thiết lập và duy trì các thực thể có quyền tương tác với dữ liệu. Trong kỷ nguyên dữ liệu lớn và các cuộc tấn công mạng phức tạp, UAM được xem là "chốt chặn" đầu tiên và quan trọng nhất để bảo vệ tài sản số của tổ chức.

Khái niệm và bản chất của tài khoản người dùng trong CSDL. Một tài khoản CSDL là một định danh duy nhất (Unique Identity) được DBMS công nhận. Khác với tài khoản hệ điều hành (Windows/Linux), tài khoản CSDL tồn tại bên trong phạm vi của công cụ quản trị dữ liệu (như MySQL, PostgreSQL). Tài khoản này là nơi lưu trữ các thông tin về: danh tính (Username), phương thức xác thực (Password/Plugin), và trạng thái (Active/Locked). Bản chất của việc quản lý tài khoản là thiết lập một "hợp đồng tin cậy" giữa người dùng và hệ thống, đảm bảo rằng mọi yêu cầu truy xuất dữ liệu đều phải thông qua bước kiểm tra danh tính nghiêm ngặt.

Vai trò của quản lý tài khoản. Vai trò của UAM bao trùm nhiều khía cạnh trong vận hành hệ thống thông tin:

- **Xác lập ranh giới bảo mật:** Tạo ra sự tách biệt giữa các cá nhân và các ứng dụng khác nhau khi cùng truy cập vào một CSDL dùng chung.
- **Kiểm soát tài nguyên:** Thông qua tài khoản, người quản trị (DBA) có thể giới hạn số lượng kết nối đồng thời, thời gian kết nối, hoặc dung lượng bộ nhớ mà một thực thể có thể sử dụng, tránh tình trạng một người dùng làm sập cả hệ thống (DoS).
- **Tạo cơ sở cho việc phân quyền (Authorization):** Nếu không có tài khoản riêng biệt, chúng ta không thể áp dụng nguyên tắc "Quyền hạn tối thiểu".
- **Đảm bảo tính không thể chối bỏ (Non-repudiation):** Khi có sự cố (ví dụ: dữ liệu bị xóa nhầm), tài khoản giúp xác định chính xác ai là người thực hiện hành động đó tại thời điểm nào.

Ý nghĩa của quản lý tài khoản đối với doanh nghiệp. Ý nghĩa lớn nhất của UAM là giảm thiểu rủi ro nội bộ. Thống kê bảo mật cho thấy hơn 60% các vụ rò rỉ dữ liệu bắt nguồn từ chính bên trong tổ chức do quản lý tài khoản lỏng lẻo.

- **Về mặt pháp lý:** Các tiêu chuẩn như GDPR, ISO 27001 hay luật An ninh mạng Việt Nam đều yêu cầu khắt khe về việc định danh người dùng truy cập dữ liệu cá nhân. Quản lý tốt tài khoản giúp doanh nghiệp tránh các khoản phạt khổng lồ.
- **Về mặt uy tín:** Một hệ thống CSDL có quản lý tài khoản chặt chẽ thể hiện sự chuyên nghiệp và cam kết bảo mật của doanh nghiệp đối với khách hàng.

Quy trình quản lý tài khoản (life-cycle management). Cần phải hiểu rằng một tài khoản có một "vòng đời" cần được quản trị:

- **Giai đoạn khởi tạo (onboarding):** Tiếp nhận yêu cầu, xác định vai trò công việc để tạo username theo đúng quy chuẩn đặt tên.
- **Giai đoạn cấp quyền:** Gán các quyền cần thiết dựa trên mô hình RBAC (Role-based Access Control).
- **Giai đoạn giám sát:** Theo dõi lịch sử đăng nhập, các lệnh SQL bất thường.
- **Giai đoạn duy trì/cập nhật:** Thay đổi mật khẩu định kỳ, cập nhật lại quyền khi nhân viên thay đổi vị trí công việc.
- **Giai đoạn thu hồi (offboarding):** Khóa hoặc xóa tài khoản ngay lập tức khi nhân viên nghỉ việc hoặc dự án kết thúc. Đây là bước thường bị bỏ quên nhất, tạo ra các "tài khoản ma" cực kỳ nguy hiểm.

Các mô hình xác thực hiện đại. Xác thực không chỉ dừng lại ở mật khẩu truyền thống, nó còn bao gồm các phương thức khác như:

- **Xác thực dựa trên password:** Phổ biến nhất nhưng dễ bị tấn công brute-force.
- **Xác thực qua chứng chỉ (certificate-based):** Sử dụng khóa SSL/TLS để xác thực máy chủ và người dùng, an toàn hơn rất nhiều.
- **Xác thực tập trung (LDAP/Active Directory):** Cho phép người dùng dùng chung tài khoản công ty để đăng nhập vào CSDL, giúp DBA quản lý tập trung một nơi.
- **Xác thực đa yếu tố (MFA):** Mặc dù khó triển khai trực tiếp trên giao thức SQL, nhưng thường được áp dụng qua các lớp proxy hoặc cổng kết nối trung gian.

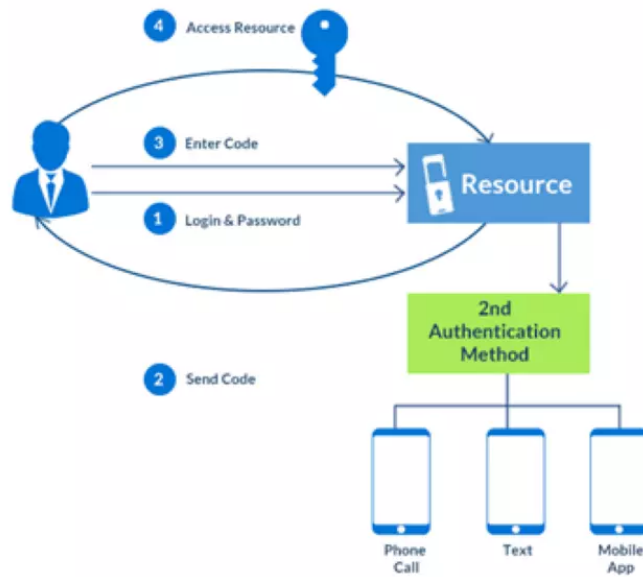
Các loại tài khoản phổ biến. Có các loại tài khoản sau tồn tại trên các hệ quản trị CSDL gồm:

- **Administrative Accounts (Root/Postgres/SA):** Tài khoản "quyền năng nhất", có quyền xóa toàn bộ CSDL. Chỉ dành cho DBA cấp cao.
- **End-user Accounts:** Tài khoản cho nhân viên (ví dụ: `ke_toan_01`, `sale_manager`).
- **Service/Application Accounts:** Tài khoản dành cho các ứng dụng (ví dụ: `web_app_user`). Các tài khoản này thường không có quyền đăng nhập từ console mà chỉ kết nối từ một địa chỉ IP server cố định.

Hãy xét ví dụ minh họa thực tế:

- **Tình huống:** Một công ty thương mại điện tử có một CSDL MySQL dùng chung.
- **Lỗ hổng về bảo mật của hệ thống:** Tất cả mọi người (lập trình viên, nhân viên kế toán, ứng dụng web) đều dùng chung tài khoản root với mật khẩu là 123456.
- **Hậu quả:** Một bạn thực tập sinh chạy nhầm lệnh `DROP DATABASE` và không ai biết ai đã làm, dữ liệu mất trắng, ứng dụng sập.

Multi Factor Authentication



Hình 6.1: Các mô hình xác thực thường sử dụng

Cách quản lý đúng cần thực hiện trên hệ thống gồm:

- Tạo tài khoản `app_web` chỉ được phép `SELECT`, `INSERT`, `UPDATE` trên các bảng sản phẩm.
- Thiết kế định danh: đặt tên theo quy tắc `[Vai trò]_[Tên]`. Ví dụ: `dev_tuan`.
- Tạo tài khoản `admin_hung` cho DBA Hùng để quản trị hệ thống.
- Tạo tài khoản `audit_ketoan` chỉ được phép `SELECT` (chỉ đọc) trên bảng hóa đơn để đối soát.
- Cấu hình tài khoản `app_web` chỉ được phép kết nối từ địa chỉ IP của web server (`10.0.0.5`).
- Thiết lập ràng buộc: luôn giới hạn host kết nối (ví dụ: `dev_tuan@'192.168.1.%'`) thay vì cho phép mọi nơi (%).
- Áp dụng chính sách mật khẩu: sử dụng mật khẩu tối thiểu 12 ký tự, bao gồm chữ hoa, chữ thường, số và ký tự đặc biệt.

6.1.2 Phân quyền người dùng trên hệ thống

Sau khi một thực thể đã được xác thực thành công thông qua quản lý tài khoản, hệ quản trị CSDL (DBMS) phải thực hiện bước tiếp theo là ủy quyền (Authorization). Phân quyền là quá trình xác định các đặc quyền (privileges) mà một tài khoản cụ thể được phép thực hiện trên các tài nguyên cụ thể của CSDL. Nếu quản lý tài khoản là "chốt chặn" đầu tiên, thì phân quyền chính là hệ thống "màng lọc" tinh vi để bảo vệ tính toàn vẹn và bảo mật của dữ liệu.

Nguyên tắc quyền hạn tối thiểu (Principle of Least Privilege - PoLP). Đây là nguyên tắc vàng trong bảo mật thông tin nói chung và CSDL nói riêng. Theo PoLP, mỗi người dùng, chương trình hoặc tiến trình chỉ được cấp những quyền hạn vừa đủ để hoàn

thành nhiệm vụ được giao, không hơn không kém.

Tại sao cần PoLP? Nếu một tài khoản ứng dụng chỉ cần đọc dữ liệu để hiển thị lên website mà lại được cấp quyền DELETE, thì một lỗ hổng SQL Injection nhỏ cũng có thể dẫn đến việc toàn bộ dữ liệu bị xóa sạch.

Triển khai: thay vì cấp quyền trên toàn bộ database, hãy cấp trên từng table; thay vì cấp quyền ALL PRIVILEGES, hãy chỉ liệt kê SELECT, INSERT.

Phân loại các nhóm quyền (Privilege Categories). Trong các DBMS hiện đại, quyền thường được chia thành hai nhóm lớn để dễ dàng quản lý:

- **Quyền hệ thống (System Privileges):** Các quyền này cho phép người dùng thực hiện các hành động có tầm ảnh hưởng trên phạm vi toàn bộ thực thể CSDL (instance). Ví dụ: CREATE ANY TABLE, BACKUP ANY DATABASE, ALTER SYSTEM, CREATE USER. Đối tượng: thường chỉ dành cho quản trị viên CSDL (DBA) hoặc các tài khoản vận hành hệ thống cấp cao.
- **Quyền đối tượng (Object Privileges):** Đây là các quyền thực thi trên các đối tượng cụ thể bên trong CSDL như bảng (table), view, hàm (function), hoặc thủ tục (stored procedure).
 - SELECT: quyền đọc dữ liệu.
 - INSERT: quyền thêm dữ liệu mới.
 - UPDATE: quyền sửa đổi dữ liệu hiện có.
 - DELETE: quyền xóa dữ liệu.
 - EXECUTE: quyền chạy một hàm hoặc thủ tục.

Mô hình kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC). Khi hệ thống có hàng trăm người dùng, việc phân quyền cho từng cá nhân là một cơn ác mộng quản trị. RBAC giải quyết vấn đề này bằng cách chèn một lớp trung gian gọi là "vai trò" (role).

- **Cơ chế:** Quyền được gán cho vai trò → vai trò được gán cho người dùng.
- **Ưu điểm:** Khi một nhân viên thay đổi bộ phận, DBA chỉ cần thu hồi role cũ và gán role mới thay vì phải rà soát và sửa đổi hàng chục lệnh GRANT/REVOKE riêng lẻ.

Ví dụ: tạo vai trò kế toán, cấp quyền SELECT trên bảng Luong. Bất kỳ nhân viên nào thuộc phòng kế toán sẽ được gán vai trò này.

Phân quyền mức độ hạt mịn (Fine-grained Access Control). Đôi khi, quyền hạn ở mức bảng vẫn là quá rộng. Chúng ta cần kiểm soát sâu hơn:

- **Quyền mức cột (column-level security):** Cho phép nhân viên nhân sự xem bảng NhanVien nhưng ẩn cột SoDienThoai hoặc MatKhau.
- **Quyền mức dòng (row-level security - RLS):** Đây là kỹ thuật cao cấp. Ví dụ: trưởng phòng kinh doanh miền Bắc chỉ được nhìn thấy các dòng dữ liệu (records) của khách hàng thuộc khu vực miền Bắc, dù tất cả dữ liệu đều nằm chung một bảng KháchHang.

Cơ chế lan truyền quyền (Grant Option). Khi cấp quyền, chúng ta có tùy chọn WITH GRANT OPTION. Điều này cho phép người nhận quyền có thể đi cấp lại quyền đó cho một người thứ ba.

Lưu ý nguy hiểm: đây là "con dao hai lưỡi". Nếu không kiểm soát chặt chẽ, quyền hạn sẽ lan truyền ngoài tầm kiểm soát của DBA ban đầu, tạo ra các lỗ hổng bảo mật dây chuyền.

Ví dụ về một thảm họa bảo mật từ việc lạm dụng quyền hạn: vào năm 2021, một ngân hàng lớn tại Mỹ đã gặp sự cố khi một nhân viên IT thuộc bộ phận hạ tầng (vốn chỉ cần quyền truy cập server) lại sở hữu tài khoản CSDL có quyền **SELECT** trên toàn bộ bảng thông tin thẻ tín dụng. Khi máy tính của nhân viên này bị nhiễm mã độc, kẻ tấn công đã sử dụng tài khoản đó để trích xuất dữ liệu của hơn 10 triệu khách hàng.

Phân tích sai lầm: vi phạm nguyên tắc PoLP. Nhân viên hạ tầng không có vai trò nghiệp vụ liên quan đến dữ liệu khách hàng nhưng lại được cấp quyền truy cập quá rộng.

Giải pháp cần khắc phục: áp dụng RBAC nghiêm ngặt và định kỳ thực hiện "audit quyền hạn" để thu hồi các quyền không còn cần thiết.

6.1.3 Thiết lập trên MySQL và PostgreSQL

Triển khai quản lý tài khoản và phân quyền trên MySQL. MySQL nổi tiếng với sự đơn giản và hiệu năng, nhưng hệ thống phân quyền của nó có một đặc điểm rất riêng biệt mà người dùng thường nhầm lẫn: danh tính người dùng luôn gắn liền với vị trí kết nối.

Khái niệm user@host độc đáo: Trong MySQL, một tài khoản không chỉ là cái tên. Nó là sự kết hợp giữa tên người dùng và địa chỉ máy khách (host). Điều này cho phép chúng ta có cùng một người dùng admin nhưng nếu kết nối từ **localhost** thì có toàn quyền, còn kết nối từ một IP lạ thì chỉ có quyền đọc. Đây là một lớp bảo mật bổ sung cực kỳ hữu ích để giới hạn phạm vi tấn công.

Quản lý mật khẩu và plugin xác thực: kể từ phiên bản 8.0, MySQL chuyển sang sử dụng **caching_sha2_password** làm plugin xác thực mặc định, mang lại tính bảo mật cao hơn nhiều so với **mysql_native_password** cũ. Khi tạo người dùng, chúng ta cần chú ý đến việc thiết lập chính sách mật khẩu (độ dài, độ phức tạp) thông qua component **validate_password**.

Triển khai roles (vai trò) trong MySQL: dù là một tính năng mới hơn (từ bản 8.0), roles trong MySQL thực chất là các tài khoản "không có quyền đăng nhập" được dùng để tập hợp các quyền. Lưu ý quan trọng: sau khi gán role cho người dùng, ta phải kích hoạt role đó bằng lệnh **SET DEFAULT ROLE** thì quyền mới có hiệu lực cho các lần đăng nhập sau.

Bảng hệ thống mysql: tất cả thông tin về tài khoản và quyền hạn được lưu trữ trong database có tên là **mysql**, cụ thể là các bảng **user**, **db**, **tables_priv**, và **columns_priv**. Việc hiểu cấu trúc các bảng này giúp quản trị viên có thể truy vấn (audit) xem ai đang có quyền gì trên toàn hệ thống một cách nhanh chóng.

Các lệnh thực thi tiêu chuẩn:

- Tạo người dùng với ràng buộc IP.
- Cấp quyền đối tượng.
- Thu hồi quyền.

```
-- Tạo người dùng với ràng buộc IP
CREATE USER 'dev_tuan'@'192.168.1.50' IDENTIFIED BY 'StrongPassword123!';
-- Cấp quyền đối tượng
GRANT SELECT, INSERT ON sales_db.* TO 'dev_tuan'@'192.168.1.50';
-- Thu hồi quyền
REVOKE INSERT ON sales_db.* FROM 'dev_tuan'@'192.168.1.50';
```

Triển khai quản lý tài khoản và phân quyền trên PostgreSQL. Nếu MySQL coi trọng sự tách biệt theo vị trí kết nối, thì PostgreSQL (Postgres) lại coi trọng tính logic, phân cấp và sự đồng nhất. Trong Postgres, mọi thực thể (người dùng, nhóm) đều được gọi chung là role.

Triết lý "role" đồng nhất: Postgres không phân biệt giữa user và group. Một role có thuộc tính LOGIN sẽ đóng vai trò là một người dùng. Một role chứa các role khác sẽ đóng vai trò là một nhóm. Cách tiếp cận này giúp việc thiết lập mô hình RBAC (kiểm soát truy cập dựa trên vai trò) trở nên cực kỳ tự nhiên và mạnh mẽ.

Phân cấp quyền phức tạp (hierarchy): đây là phần người dùng dễ mắc lỗi nhất khi sử dụng. Trong Postgres, để truy cập được một bảng, người dùng cần ba cấp quyền:

- Quyền kết nối vào database (CONNECT).
- Quyền sử dụng schema chứa bảng đó (USAGE).
- Quyền thao tác trên bảng cụ thể (SELECT, UPDATE...). Nếu ta chỉ GRANT SELECT ON TABLE mà quên USAGE ON SCHEMA, người dùng sẽ nhận thông báo lỗi "Permission denied" ngay lập tức.

Cơ chế xác thực qua pg_hba.conf: khác với MySQL quản lý host ngay trong câu lệnh SQL, Postgres sử dụng một file cấu hình hệ thống gọi là pg_hba.conf (Host-Based Authentication). File này định nghĩa: "Ai, từ đâu, vào database nào, sử dụng phương thức xác thực gì (MD5, Scram-sha-256, Trust, Reject)". Đây là lớp bảo vệ vòng ngoài cực kỳ nghiêm ngặt trước khi lệnh SQL được xử lý.

Quyền mặc định (default privileges): một điểm ưu việt của Postgres là lệnh ALTER DEFAULT PRIVILEGES. Nó cho phép người dùng thiết lập rằng: "trong tương lai, bất kỳ bảng nào được tạo ra bởi admin, thì role readonly_group sẽ tự động có quyền SELECT". Điều này giải quyết bài toán quản trị quyền cho các đối tượng chưa tồn tại.

Các lệnh thực thi tiêu chuẩn:

- Tạo role nhóm: CREATE ROLE readonly_role;
- Cấp quyền schema & table: GRANT USAGE ON SCHEMA public TO readonly_role;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_role;
- Gán người dùng vào nhóm: GRANT readonly_role TO user_a;

6.2 Thực hành quản trị CSDL

6.2.1 Cài đặt và triển khai DB với docker compose

Để cài đặt và triển khai hai DBMS là MySQL và PostgreSQL thông qua docker compose, ta cần tạo file cấu hình docker-compose.yml như thông thường. Để có thể quản trị hai DBMS này, ta thường sử dụng các công cụ hỗ trợ giao diện người dùng để thao tác trực quan và dễ dàng hơn thay vì sử dụng command line và cấu hình thông qua các lệnh trực tiếp. Với MySQL, công cụ quản lý thường dùng là phpMyAdmin, còn đối với PostgreSQL, ta có thể sử dụng pgAdmin.

Dưới đây là mã nguồn của file docker compose chứa các service cần thiết để cài đặt và quản trị hai DBMS nói trên.

```
services:
  mysql:
    image: mysql:8.0
```

```
container_name: mysql_db
restart: always
environment:
  MYSQL_ROOT_PASSWORD: rootpassword
  MYSQL_DATABASE: myapp
  MYSQL_USER: myuser
  MYSQL_PASSWORD: mypassword
ports:
  - "3306:3306"
volumes:
  - mysql_data:/var/lib/mysql

postgres:
  image: postgres:16
  container_name: postgres_db
  restart: always
  environment:
    POSTGRES_DB: myapp
    POSTGRES_USER: myuser
    POSTGRES_PASSWORD: mypassword
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data

phpmyadmin:
  image: phpmyadmin/phpmyadmin:latest
  container_name: phpmyadmin
  restart: always
  environment:
    PMA_HOST: mysql
    PMA_PORT: 3306
    PMA_USER: myuser
    PMA_PASSWORD: mypassword
  ports:
    - "8080:80"
  depends_on:
    - mysql

pgadmin:
  image: dpage/pgadmin4:latest
  container_name: pgadmin
  restart: always
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@example.com
    PGADMIN_DEFAULT_PASSWORD: adminpassword
  ports:
    - "8081:80"
  depends_on:
    - postgres

volumes:
  mysql_data:
  postgres_data:
```

Khởi tạo các container với câu lệnh quen thuộc:

```
$docker compose up -d
```

Kiểm tra xem các container đã được khởi tạo thành công hay chưa, các dịch vụ đã hoạt động và cổng dịch vụ đã được public như trong thiết lập của file cấu hình chưa. Nếu mọi thứ đều hoạt động tốt, ta có thể truy cập phpMyAdmin (MySQL) qua URL: <http://localhost:8080>, thông tin đăng nhập bằng myuser/mypassword.

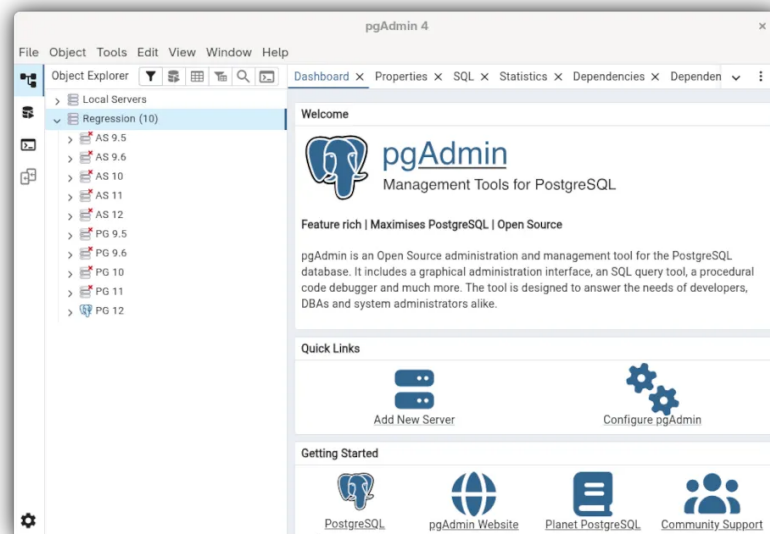
Với pgAdmin (PostgreSQL), ta truy cập qua địa chỉ <http://localhost:8081>, sử dụng thông tin đăng nhập bằng tài khoản: admin@example.com/adminpassword.

Sau khi đăng nhập pgAdmin, bạn cần tạo kết nối mới đến server PostgreSQL với thông tin:

- Host: postgres
- Port: 5432
- Username: myuser
- Password: mypassword

Lưu ý quan trọng:

- Đổi mật khẩu mặc định trong file compose trước khi sử dụng thực tế.
- Các cổng 8080 và 8081 chỉ nên mở khi cần thiết và nên giới hạn truy cập (ví dụ dùng reverse proxy hoặc firewall).
- Dữ liệu vẫn được lưu trữ an toàn trong volume như trước.



Hình 6.2: Giao diện công cụ pgAdmin

6.2.2 Thiết lập và cấu hình CSDL trên phpmyadmin

Yêu cầu của bài thực hành:

- Quản lý người dùng và phân quyền theo nguyên tắc Least Privilege.
- Thiết lập chính sách mật khẩu mạnh và xác thực hai yếu tố (nếu có).
- Bảo vệ chống SQL Injection.
- Mã hóa dữ liệu nhạy cảm (cột và kết nối).
- Kích hoạt logging/auditing để theo dõi hoạt động.

- Nhận diện và khắc phục các cấu hình bảo mật yếu phổ biến.

Chuẩn bị môi trường trước khi thực hành: truy cập phpMyAdmin: <http://localhost:8080> như trên, sau đó đăng nhập bằng tài khoản root (mặc định không có mật khẩu hoặc là root).

Tạo database mẫu: `university_db` chứa các bảng:

- `users` (id, username, password_hash, email, role, ssn — dữ liệu nhạy cảm)
- `students` (id, name, dob, grade, address)
- `grades` (student_id, course, score)

Trong phpMyAdmin → tab Databases → tạo database tên `university_db`. Chọn database → tab Import → import file script mẫu.

```
USE university_db;

CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50),
  password_hash VARCHAR(255),
  email VARCHAR(100),
  role VARCHAR(20),
  ssn VARCHAR(20)
);

CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  dob DATE,
  address VARCHAR(255)
);

CREATE TABLE grades (
  id INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT,
  course VARCHAR(50),
  score FLOAT
);

-- Insert dữ liệu mẫu
INSERT INTO users (username, password_hash, email, role, ssn) VALUES
('admin', 'hash123', 'admin@uni.edu', 'admin', '123-45-6789');
```

Nên backup database ban đầu (để khôi phục nếu sai).

Phần 1: Quản lý người dùng & phân quyền

- Trong phpMyAdmin, click User accounts (ở menu trên cùng).
- Click Add user account.
- Tạo các tài khoản sau: `app_user` (host: localhost) – password `AppPass123!`; `report_user` (host: localhost) – password `ReportPass456!`.
- Sau khi tạo, click vào tên user → tab Database privileges.
- Gán quyền: `app_user`: chọn database `university_db` → tick `SELECT`, `INSERT`, `UPDATE` (không tick `DELETE`, `DROP`, `ALTER...`); `report_user`: chỉ tick `SELECT`.
- Nhiệm vụ cần thực hành:
 - Đăng xuất root → đăng nhập bằng `app_user` và `report_user`.

- Thử thực hiện các lệnh không được phép (ví dụ: `DROP TABLE`) → quan sát thông báo lỗi.
- Quay lại root → thu hồi bớt quyền và kiểm tra lại.

Phần 2: Chính sách mật khẩu

- Truy cập vào User accounts → chọn user → Edit privileges → Change password.
- Hãy thử đặt mật khẩu yếu (ví dụ: 123) → quan sát phpMyAdmin có cảnh báo không.
- Vào tab SQL, chạy lệnh thiết lập chính sách (phpMyAdmin hỗ trợ):

```
SET GLOBAL validate_password.policy = MEDIUM;  
SET GLOBAL validate_password.length = 12;
```

- Tiếp theo, cấu hình để buộc user đổi mật khẩu lần đầu và đặt thời hạn hết hạn 90 ngày (qua SQL)? Hãy thực hiện.

Phần 3: Mã hóa dữ liệu

Trong phpMyAdmin, chọn bảng `users` → Structure → Add column → thêm cột `ssn_enc` kiểu `VARBINARY(255)`. Chạy lệnh mã hóa trong tab SQL:

```
UPDATE users  
SET ssn_enc = AES_ENCRYPT(ssn, 'SuperSecretKey123!');
```

Kiểm tra: hãy xem dữ liệu và so sánh cột `ssn` và `ssn_enc`.

Để thực hiện cấu hình nâng cao hơn, ta sẽ thiết lập kết nối SSL (nâng cao): hãy tìm hiểu để thực hiện việc bật `require_secure_transport = ON` trong file `my.ini` của XAMPP, sau đó restart MySQL. Thực hiện kiểm tra với Wireshark để bắt gói tin và phân tích để thấy được hiệu quả khi tạo kết nối bảo mật.

Phần 4: Thiết lập logging & auditing

Vào tab SQL, bật logging:

```
SET GLOBAL general_log = 1;  
SET GLOBAL general_log_file = 'C:/xampp/mysql/data/general.log';
```

Đường dẫn file log có thể thay đổi cho phù hợp với hệ điều hành. Tiếp theo:

- Thực hiện một số thao tác → xem file log trong thư mục data của MySQL.
- Tìm hiểu cách tạo trigger đơn giản ghi log thay đổi điểm số (qua tab SQL).

Phần 5: Nội dung thực hành mở rộng

- Export database và import lại với mã hóa (sử dụng phpMyAdmin Export).
- Thử tấn công brute-force (sử dụng công cụ như Hydra) và quan sát log.

6.2.3 Thiết lập và cấu hình CSDL với PostgreSQL

Mục tiêu bài thực hành:

- Thực hành bảo mật PostgreSQL chủ yếu qua giao diện pgAdmin.
- Áp dụng nguyên tắc Least Privilege với Role.
- Sử dụng Row Level Security (RLS) – điểm mạnh của PostgreSQL.

- Phòng chống SQL Injection.
- Mã hóa dữ liệu nhạy cảm (pgcrypto).
- Bật logging/auditing cơ bản.
- Nhận diện và khắc phục lỗ hổng phổ biến.

Chuẩn bị môi trường:

- Cài PostgreSQL + pgAdmin 4 (có thể dùng PostgreSQL Installer cho Windows hoặc gói trên Ubuntu).
- Mở pgAdmin → đăng nhập server mặc định (thường là user postgres).
- Tạo database mẫu:
 - Right-click Databases → Create → Database → đặt tên `university_db`.
 - Click vào database → tab Query Tool → chạy script tạo bảng (dưới đây).
 - Copy-paste đoạn mã tạo bảng hoặc nhập nội dung vào Query Tool:

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50),  
    password_hash VARCHAR(255),  
    email VARCHAR(100),  
    role VARCHAR(20),  
    ssn VARCHAR(20)  
);  
  
CREATE TABLE students (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    dob DATE,  
    address VARCHAR(255)  
);  
  
CREATE TABLE grades (  
    id SERIAL PRIMARY KEY,  
    student_id INT,  
    course VARCHAR(50),  
    score FLOAT  
);  
  
-- Insert dữ liệu mẫu  
INSERT INTO users (username, password_hash, email, role, ssn)  
VALUES ('admin', 'hash123', 'admin@uni.edu', 'admin', '123-45-6789');
```

Cài extension cần thiết trong Query Tool:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Phần 1: Quản lý role & phân quyền (Least Privilege)

- Trong Browser (bên trái), mở rộng Login/Group Roles.
- Right-click Login Roles → Create → Login Role.
- Tạo các role như sau: `app_user`: Definition tab → nhập Password `AppPass123!`; `report_user`: Definition tab → nhập Password `ReportPass456!`.
- Sau khi tạo, right-click role → Properties → tab Privileges hoặc sử dụng Grant Wizard: right-click database `university_db` → Grant Wizard.

- Chọn Grantee là `app_user` → tick `CONNECT`, `SELECT`, `INSERT`, `UPDATE` (không cho `DELETE`, `DROP`...). Tương tự cho `report_user`: chỉ `SELECT`.
- Các yêu cầu của bài thực hành cần thực hiện:
 - Tạo mới kết nối trong pgAdmin với user `app_user` và `report_user`.
 - Thử truy vấn bảng không được phép → quan sát lỗi và viết lại báo cáo.
 - Quay lại user `postgres` → thu hồi quyền thừa qua Grant Wizard và kiểm tra lại.

Phần 2: Authentication & chính sách mật khẩu

- Right-click user (ví dụ `app_user`) → Properties → tab Definition → thay đổi password.
- Thử đặt mật khẩu yếu (123) → quan sát.
- Trong Query Tool, chạy lệnh thiết lập (pgAdmin hỗ trợ) sau đây:

```
ALTER USER app_user PASSWORD EXPIRE;
```

Nhiệm vụ cần thực hiện: buộc user đổi mật khẩu lần đầu và ghi chú cách cấu hình `pg_hba.conf` (để dùng SCRAM-SHA-256) qua file config.

Ý nghĩa:

- File `pg_hba.conf` là file Host-Based Authentication của PostgreSQL. Nó quyết định phương thức xác thực (authentication method) khi người dùng kết nối từ đâu, dùng database nào.
- SCRAM-SHA-256 là phương thức xác thực an toàn nhất hiện nay của PostgreSQL (tốt hơn MD5 rất nhiều). Nó bảo vệ mật khẩu khỏi bị nghe lén (eavesdropping) và tấn công replay.

Cách cấu hình (thông qua file config):

Tìm file `pg_hba.conf`:

- Trong pgAdmin: right-click Server → Properties → tab Connection → xem Configuration File hoặc Data Directory.
- Thường nằm ở: `C:\Program Files\PostgreSQL\16\data\pg_hba.conf` (Windows) hoặc `/etc/postgresql/16/main/pg_hba.conf` (Linux).

Mở file bằng Notepad / VS Code (chạy với quyền Administrator). Thay đổi hoặc thêm dòng sau (ưu tiên SCRAM):

#	TYPE	DATABASE	USER	ADDRESS	METHOD
host	all	all	all	127.0.0.1/32	scram-sha-256
host	all	all	all	:::1/128	scram-sha-256
host		university_db	app_user	0.0.0.0/0	scram-sha-256

Trong đó:

- `scram-sha-256`: phương thức mạnh.
- `md5`: cũ và kém an toàn (nên tránh).
- `trust`: rất nguy hiểm (không cần mật khẩu).

Lưu file cấu hình và khởi động lại PostgreSQL Server: trong pgAdmin: right-click Server → Restart.

Bảng 6.1: Tóm tắt lý do bảo mật cấu hình `pg_hba.conf`

Yếu tố	Mục đích bảo mật	Mức độ quan trọng
Buộc đổi mật khẩu lần đầu	Ngăn mật khẩu mặc định bị lạm dụng	Cao
SCRAM-SHA-256	Bảo vệ mật khẩu khi truyền qua mạng	Rất cao
<code>pg_hba.conf</code>	Kiểm soát ai được kết nối từ đâu, bằng cách nào	Rất cao

Tóm tắt lý do bảo mật:

Phần 3: Logging & auditing

Trong Query Tool, bật logging cơ bản:

```
ALTER SYSTEM SET log_statement = 'all';
ALTER SYSTEM SET log_connections = on;
```

Tiếp theo, khởi động lại server PostgreSQL. Thực hiện thao tác xem log trong thư mục data của PostgreSQL (hoặc pgAdmin → Dashboard).

Yêu cầu nâng cao: tìm trên GitHub và cài extension `pgAudit` và cấu hình sử dụng công cụ đó để thực hiện thao tác xem và quản lý log với PostgreSQL.

Câu hỏi thảo luận

Câu 1: Tại sao không nên đặt tên tài khoản là `admin` hay `manager`? (Gợi ý: liên quan đến tấn công dò tìm tên người dùng).

Câu 2: Một nhân viên IT lâu năm chuyển sang bộ phận Marketing. Bạn sẽ làm gì với tài khoản CSDL của anh ta? Giải thích lý do tại sao không nên chỉ xóa quyền mà nên thu hồi toàn bộ tài khoản cũ.

Câu 3: Hãy liệt kê ít nhất 3 tiêu chí để đánh giá một hệ thống quản lý tài khoản CSDL là "an toàn".

Câu 4: So sánh các cơ chế mã hóa MD5 và SCRAM-SHA-256 trong CSDL PostgreSQL.

Bài thực hành

Bài 1. Quy trình phân quyền tài khoản trên DBMS